

StratRoom

How to Access the Server

A practical, illustrated guide to reaching the StratRoom production and local server - web, API, SSH, database and Docker.

Prepared August 2026 • Version 1.0 • Internal documentation

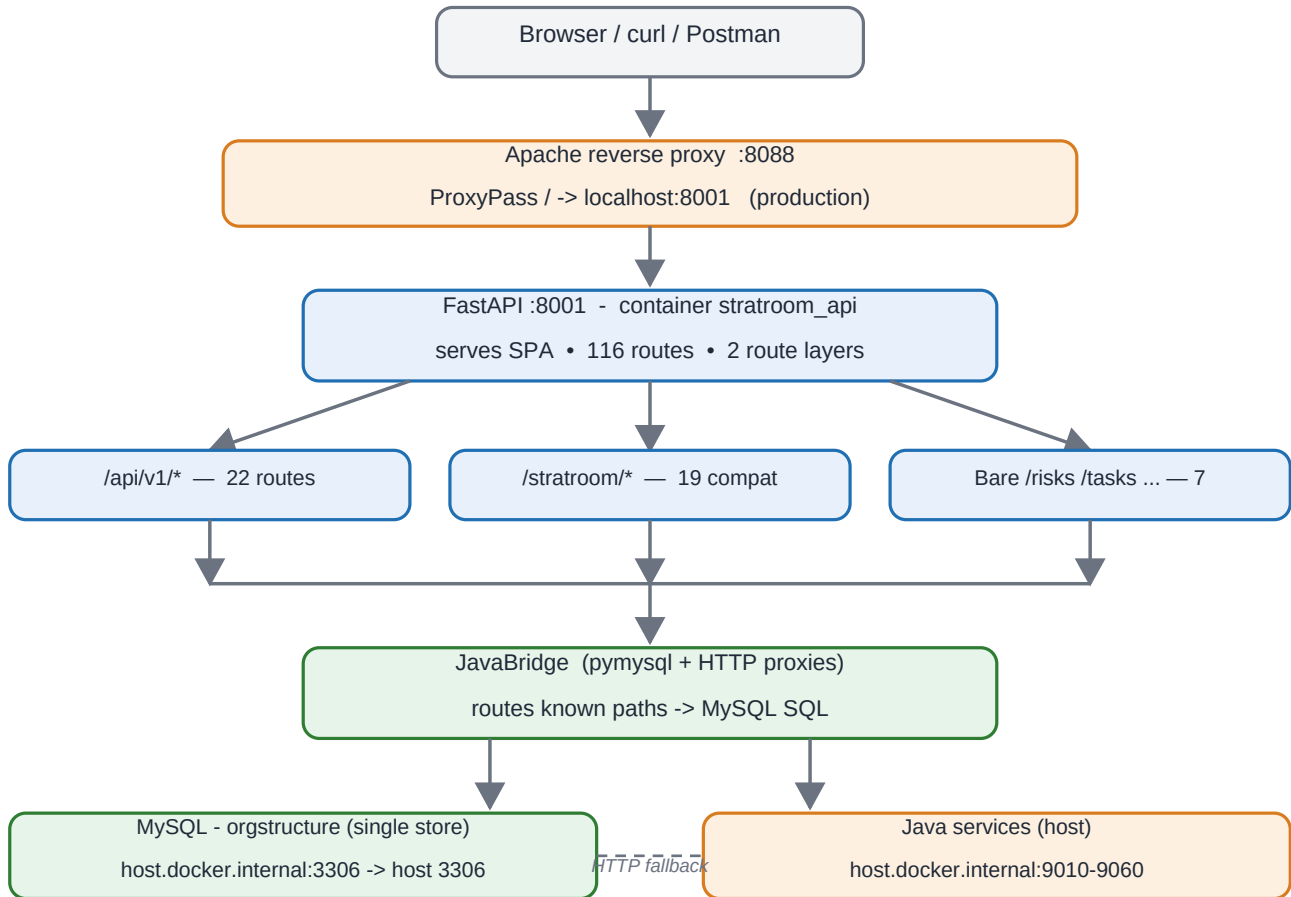
1. Overview

StratRoom is a full-stack, AI-powered enterprise governance dashboard. The production server is hosted at **103.191.132.36**: an **Apache reverse proxy on port 8088** forwards all traffic to a **FastAPI** application on port **8001**, which runs inside the Docker container **stratroom_api**. Every read and write is persisted to a single **MySQL** store, schema **orgstructure**. PostgreSQL was fully removed in July 2026.

There are five ways to reach the server. Use the quick reference below, then read the matching section for the detailed steps and flow chart.

Access method	Entry point	Use when
Production web (browser)	http://103.191.132.36:8088	Human users; demo alias demo.stratroom.io:8088
Local web (Docker dev)	http://localhost:8001	Local development; Swagger docs at /docs
API (curl / scripts)	http://103.191.132.36:8088/api/v1/...	JWT bearer token required after login
SSH to host	plink -P 55004 -l root 103.191.132.36	Ops/admin; password via env SSH_PASS
MySQL database	tunnel 127.0.0.1:3307 -> host 3306	orgstructure schema; mysql/mysqldump tools
Docker container	docker exec -it stratroom_api bash	Inside the app; PYTHONPATH=/app/backend

FLOW CHART 1 — Full request path: client -> server -> data



Production traffic enters only via Apache :8088. Local dev hits FastAPI directly on :8001.

AI/ML endpoints (/ai/chat, /agents/chat, /ml/*) call external LLM providers and XGBoost - not MySQL.

Chart 1 - The complete path from a client to the data. Production traffic always enters via Apache :8088; local development goes straight to FastAPI :8001.

2. Web (browser) access

2.1 Production

- Open **http://103.191.132.36:8088** (or **http://demo.stratroom.io:8088**) in any browser.
- Apache :8088 forwards the request with **ProxyPass / -> localhost:8001** to FastAPI.
- FastAPI's **serve_frontend()** returns the single-file SPA `31may_index.html` (1.5 MB, 20 dashboard modules).
- No credentials are needed to load the page - you sign in on the login screen (see Chart 2).

2.2 Local development (Docker)

```

docker compose up -d --build # builds + starts container stratroom_api curl
http://localhost:8001/health # liveness -> {"status":"ok"} curl http://localhost:8001/ready #
readiness -> database + mysql status browser: http://localhost:8001 # SPA browser:
http://localhost:8001/docs # Swagger UI (only when ENABLE_DOCS=true)
  
```

The Docker Compose file maps host port **8001** to container port **8000**. **Important:** the Apache document root `/var/www/stratroom-ai/index.html` is **not** what is served in production - the live page comes from inside the container.

3. Authentication & login

StratRoom has two live login paths, both verified in production. The SPA login form uses the password-verified path; API tools usually use the passwordless path.

Path	Endpoint	Body	Notes
Passwordless	POST /api/v1/auth/login	{"email":"admin@stratroom.com"}	Checks MySQL users, falls back to JavaBridge /userList. Returns JWT access_token.
Password-verified	POST /auth/login	{"email":"admin@stratroom.com","password":"changeme"}	bcrypt check against MySQL users.hash_password. This is what the SPA form calls.

FLOW CHART 2 — Login & JWT authentication flow

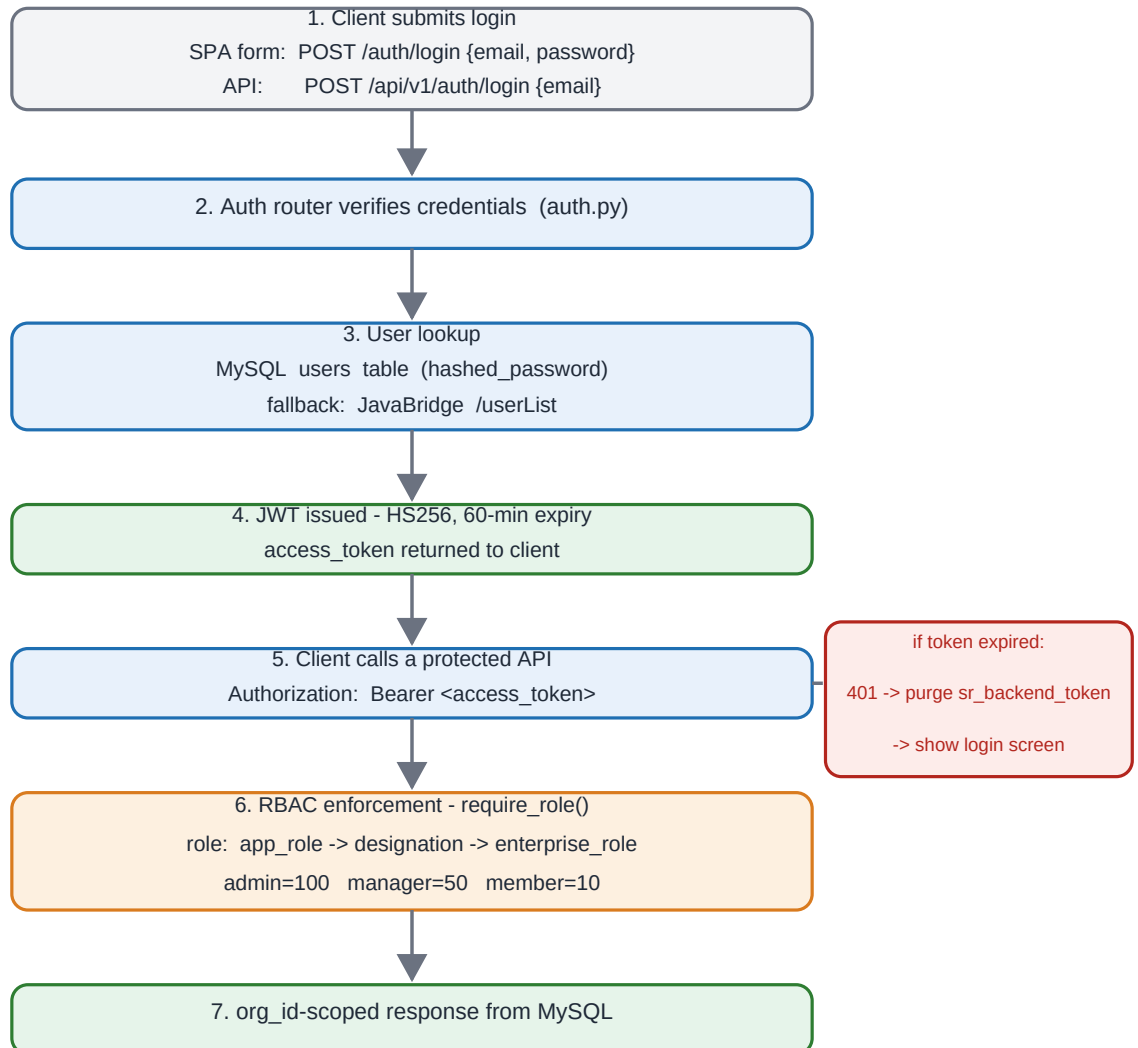


Chart 2 - Login produces a signed JWT (HS256, 60 minutes). Every later API call carries it in the Authorization header; require_role() resolves admin / manager / member. On expiry the SPA purges its stored tokens and returns to the login screen.

4. API access (curl)

The API is a FastAPI service exposing **116 routes**. The pattern is always the same: **login -> get token -> call endpoint with the Bearer token**. Below are working commands from the production host URL (use `localhost:8001` locally).

4.1 Health and readiness

```
curl.exe http://103.191.132.36:8088/health # {"status":"ok","version":"1.0.0",...} curl.exe
http://103.191.132.36:8088/ready # {"status":"ready","database":"ok","mysql":"ok",...}
```

4.2 Login and capture the JWT

```
# PowerShell (passwordless v1 path) $token = (curl.exe -s -m 10 -X POST
http://103.191.132.36:8088/api/v1/auth/login ` -H "Content-Type: application/json" ` -d
'{"email":"admin@stratroom.com"}' | ConvertFrom-Json).access_token # bash TOKEN=$(curl -s -X POST
http://localhost:8001/api/v1/auth/login \ -H "Content-Type: application/json" \ -d
'{"email":"admin@stratroom.com"}' \ | python -c "import sys,json;
print(json.load(sys.stdin)['access_token'])")
```

4.3 Call authenticated endpoints

```
# Bare route -> MySQL scorecard_kpis curl.exe -s -m 10 http://103.191.132.36:8088/scorecards -H
"Authorization: Bearer $token" # Compat route -> MySQL via JavaBridge curl.exe -s -m 10
"http://103.191.132.36:8088/stratroom/riskList?pageId=3196" -H "Authorization: Bearer $token" # v1
endpoints -> login, dashboard, org, incidents, tasks, AI insights... curl.exe -s -m 10
http://103.191.132.36:8088/api/v1/dashboard/kpis -H "Authorization: Bearer $token"
```

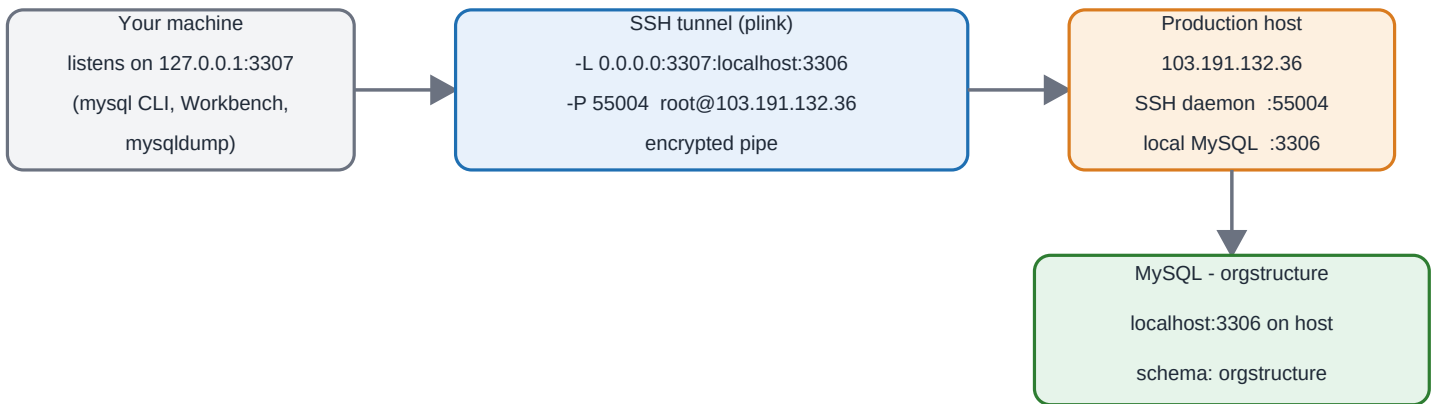
Route families: **/api/v1/*** (22 routes), **/stratroom/*** compat (19 routes, MySQL via JavaBridge), and 7 bare routes (`/risks`, `/tasks`, `/scorecards`, `/budgets`, `/org`, `/meetings`, `/dashboard/stats`). All require the member role; unauthenticated calls get **401**.

5. SSH access to the host

For administration (deploys, tunnels, service checks) you SSH to the production host. All SSH scripts read the password from the environment variable **SSH_PASS** - it is never hard-coded.

```
$env:SSH_PASS = "" # never hard-code it in a script # interactive session (putty/plink, port 55004,
user root) plink -ssh -P 55004 -l root -pw "$env:SSH_PASS" 103.191.132.36 # open the MySQL tunnel
(bind 0.0.0.0 - required for Docker Desktop host.docker.internal) plink -ssh -P 55004 -l root -pw
"$env:SSH_PASS" -L 0.0.0.0:3307:localhost:3306 -N 103.191.132.36 # from the host you can reach the
container directly ssh root@103.191.132.36 'docker ps' # shows stratroom_api (8001 -> 8000)
```

FLOW CHART 3 — SSH tunnel -> MySQL (from your Windows machine)



Any tool that connects to 127.0.0.1:3307 on your machine actually reaches the production MySQL host on port 3306.

Chart 3 - The plink tunnel maps your local port 3307 to the host's MySQL on 3306, so local tools can talk to the production database over an encrypted SSH connection.

6. MySQL database access

All data lives in MySQL schema **orgstructure** on the host (port 3306). The container reaches it through **host.docker.internal:3306**; you reach it from your machine through the tunnel above.

```
# with the tunnel running, connect as if MySQL were local mysql -h 127.0.0.1 -P 3307 -u root -p orgstructure # dump / restore through the same tunnel mysqldump -h 127.0.0.1 -P 3307 -u root -p orgstructure > backup_2026.sql mysql -h 127.0.0.1 -P 3307 -u root -p orgstructure < backup_2026.sql
```

Restoring the 'Document from Gowtham' dump: the file `D:\project001\Document from Gowtham` (approx. 707 MB, no extension) is a MySQL dump - its header starts with `-- MySQL`. Import it with the command above, replacing `backup_2026.sql` with that file path. Treat it as a restore of the `orgstructure` schema.

Key tables	What they hold
users	Auth + hashed_password (bcrypt); login identity
employee_details	Org tree via parent_emp_id (org_members is empty); 65 rows
user_role_management	Designation -> RBAC role mapping; 65 rows
score_card	108 scorecard definitions (names/weights/dates)
scorecard_kpis	408 KPI values (target/actual/status) - different data from score_card
tasks / risks / incidents / meetings / initiatives / audit_findings	Application module data
agent_conversations / agent_messages / ai_agent_runs / ai_memory	AI layer persistence
risk_details / budget_detail / compliance_details ...	JavaBridge legacy tables

7. Docker access

The app runs as a single container named **stratroom_api** (FastAPI on port 8000 inside, mapped to 8001 on the host). Use these commands for daily operations.

```
docker ps # confirm stratroom_api is Up (healthy) docker logs stratroom_api --tail 50 -f # live logs; JSON access logs - grep for 4xx/5xx docker exec -it stratroom_api bash # shell inside the container python -c "import app.main" # PYTHONPATH=/app/backend; imports use app.* docker compose up -d --build # rebuild after backend code changes docker cp /opt/stratroom-new/frontend/31may_index.html stratroom_api:/app/frontend/31may_index.html # frontend-only deploy - no restart needed; the live page updates immediately
```

Backend deploys use **deploy.py** (copies 41 files, restarts the container, exits on error). Frontend-only changes use the **docker cp** line above - the Apache doc root is cosmetic only.

8. Credentials & health checks

8.1 Default users (org_id = 1)

Email	Password	Role	Notes
admin@stratroom.com	changeme	admin	Full CRUD on all org data
admin@test.com	changeme	member	Assigned data only

8.2 Health endpoints

Endpoint	Purpose	Expected response
GET /health	Liveness probe	{"status":"ok","version":"1.0.0","environment":"production"}
GET /ready	Readiness (DB + MySQL)	{"status":"ready","database":"ok","mysql":"ok",...}
GET /metrics	AI counters, uptime, version	{"uptime_seconds":N,"ai_metrics":{...},...}

9. Troubleshooting & gotchas

- **Rate limiter is in-memory per-process.** The container must run with `--workers 1`; more workers breaks limiting (120 req/min global, 30/min AI).
- **JWT_SECRET must be a static 64-char hex.** If unset it is auto-generated and every token dies on restart - logins break until users sign in again.
- **CORS_ORIGINS** must list the real frontend domain, or browser calls from other origins are blocked.
- **Compat routes (/stratroom/*) require a JWT.** Unauthenticated requests get 401, not fallback data.
- **Database is MySQL only.** The PostgreSQL container was removed in July 2026; db.py yields None gracefully and is not used for real I/O.
- **Different data, not duplicates:** score_card (definitions) vs scorecard_kpis (values); audit_findings is the audit table, not audit.
- **Org tree** uses employee_details.parent_emp_id because the org_members table is empty.
- **Dashboard summary** returns 0 for any bridge data that fails rather than crashing.
- **Frontend is a single 1.5 MB file** (31may_index.html). No build step - edit it directly.